

Add Identity provider [info](#)

Provider details

Provider type [Info](#)

☐ SAML
Establish trust between your AWS account and a SAML 2.0 compatible Identity Provider such as Shibboleth or Active Directory Federation Services.

☒ OpenID Connect
Establish trust between your AWS account and Identity Provider services, such as Google or Salesforce.

Provider URL
Specify the secure OpenID Connect URL for authentication requests.

Maximum 255 characters. URL must begin with "https"

Audience [Info](#)
Specify the client ID issued by the Identity provider for your app.

Maximum 255 characters. Use alphanumeric or "_.-/" characters.

Figure 2: Adding identity provider

Step 1: Create the OIDC identity provider in AWS: The foundation of a secretless architecture is establishing a ‘trust relationship’ between AWS and GitHub. By creating an OIDC identity provider, you are essentially telling AWS to recognise and trust digital identity tokens signed by GitHub’s official servers. To do this, navigate to the *IAM Console*, select *Identity providers*, and click on *Add provider*. Choose *OpenID Connect* as the provider type and enter *https://token.actions.githubusercontent.com* as the provider URL. For the audience, use *sts.amazonaws.com*. This ensures your AWS environment can securely communicate with GitHub’s identity service as shown in Figure 2.

Step 2: Create the secretless IAM role: Once the trust is established, you need to create an IAM role that the GitHub runner will ‘assume’ to perform tasks. The security of this setup relies entirely on the trust policy attached to this role. Unlike traditional roles, this policy uses a *StringLike* condition to enforce strict Zero Trust boundaries. It ensures that AWS only grants access if the token’s ‘sub’ (subject) claim matches your specific GitHub organisation and repository. By using a condition like *repo:arundotcom/sql-learning:**, you ensure that even if another user knows your Role ARN, they cannot use it from their own GitHub account, as their identity token will not match your

specific repository claim. Given below is the code snippet that could be used to create the IAM role:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::AWS-ACCOUNT-ID:oidc-provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
          "token.actions.githubusercontent.com:sub": "repo:NAME-OF-THE-ORG/NAME-OF-THE-REPO:ref:refs/heads/main"
        }
      }
    }
  ]
}
```

Step 3: Update the GitHub workflow

The final step is to configure your GitHub Actions YAML file to request the identity token. You must explicitly set the *id-token: write*

permission in your workflow; without this, GitHub will not generate the OIDC token required for the handshake. In the deployment steps, you use the official *aws-actions/configure-aws-credentials* action, providing only the Role ARN and the AWS Region. Because the handshake happens automatically in the background, you no longer need to store or reference *AWS_ACCESS_KEY_ID* or *AWS_SECRET_ACCESS_KEY* in your repository secrets. When the job runs, the logs will confirm that the runner has assumed the role successfully as shown in Figure 3, proving your pipeline is now completely secretless. The code snippet given below could be used to write the YAML file:

```
name: OIDC Test
on: [push]

permissions:
  id-token: write # This is mandatory for OIDC
  contents: read

jobs:
  test-aws-connection:
    runs-on: ubuntu-latest
    steps:
      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::549368061164:role/GitHub-
```